



# Projet Annuel :

Connected Neighbours "Voisins, services et bonne humeur"

# Sommaire :

1 – Description du projet

2 – Présentation de l'infrastructure et des technologies

3 – Charte graphique

# 1 – Description du projet

## Projet : « Petits Secrets Entre Voisins »

**Vision du projet** L'objectif de "Petits Secrets Entre Voisins" est de fournir une plateforme centralisée, sécurisée et extensible, dédiée à la vie de quartier. Elle permet aux résidents d'interagir, de s'entraider via des services, et de simplifier leurs démarches administratives locales tout en renforçant le tissu social.

## Fonctionnalités Web Principales :

- **Cartographie intelligente** : Définition des limites géographiques du quartier par l'administrateur via un outil de dessin **React-Leaflet et Geoman**. Les routes et connexions géographiques seront modélisées avec Neo4j pour une meilleure analyse spatiale.

Pourquoi le choix de **Leaflet** ? C'est une solution Open-source et classique donc fortement documentée et c'est également une librairie React.

Pourquoi le choix de **Geoman** ? Ce plugin nous permet de faire des cartes beaucoup plus interactives avec plus de possibilités que Leaflet.draw.

- **Services et Contrats Automatisés** : Un système de petites annonces pour offrir ou demander des services. Les services payants utilisent un système de points virtuels et déclenchent la création automatique d'un contrat généré via des workflows **N8N**.

**Outil N8N** : Il nous permet d'automatiser certaines tâches. Nous avons la possibilité d'ajouté de nouveaux modules sans la modification de notre code (exemple : envoi de mail, facturation) et nous pouvons également l'héberger sur notre serveur ou dans un cloud.

- **Gestion Documentaire & Signatures** : On pourra importer, exporter et annoter des documents au format PDF. Les contrats intègrent un système de signature numérique sécurisée et archivée.

## Frontend React

Pour l'affichage du contrat généré par N8N, on utilisera la librairie **react-pdf**.

Pour la signature de l'utilisateur, elle se fera avec **react-signatures-canvas**.

La validation de la signature se fera avec l'outil d'authentification à multi-facteur **Keycloak** (description en dessous)

## Backend Nodejs

L'utilisation de la librairie **pdf-lib** pour la manipulation des pdf

La signature numérique (PKI) avec la librairie **node-signpdf** (signature cryptographique qui permet d'assurer la véracité du document)

## MongoDB

Archivage du contrat

- **Vie de Quartier & Événements** : La création et la gestion d'activités communautaires. Une fonction de point par rapport à l'activité des voisins (plus un voisin participe à des événements ou aide à la vie du quartier, plus il aura des points et ces points lui permettra d'obtenir un certain statut au sein du quartier comme un badge). Le système inclut des suggestions automatiques d'événements propulsées par l'analyse des interactions passées sous **Neo4j**.

- **Communication & Démocratie** : Une messagerie multimédia sécurisée (texte, vocal) en temps réel et un module de vote paramétrable pour les décisions de la vie quotidienne. L'application sera entièrement multilingue grâce à la librairie **i18next**.

## Administration & Client Lourd (Mode Hors-Ligne) :

Une application Desktop en **Java** destinée aux administrateurs pour gérer les incidents signalés et analyser les statistiques de participation.

Conçue en approche **offline-first**, elle permet la consultation et l'ajout de données sans connexion, avec une synchronisation automatique (et gestion des conflits) dès le retour du réseau.

**JavaFX Charts** : Utilisation des bibliothèques natives de **JavaFX** pour générer des graphiques (PieChart, BarChart, LineChart) donc il nous servira pour les différents statistiques.

**Base locale** : Interrogation de la base de données locale **H2** présente sur l'ordinateur de l'administrateur. Ainsi, la génération des graphiques est instantanée et fonctionne parfaitement en mode hors-ligne.

## Fonctionnalités détaillées par interface :

*Description Fonctionnelle : Front-End Administrateur (React)*

Le back-office, développé en React, est le centre de contrôle de l'application "Petits Secrets Entre Voisins". Il permet aux modérateurs et administrateurs de gérer l'activité de la plateforme de manière centralisée et sécurisée. L'interface est structurée autour des modules suivants :

### 1. Module de Cartographie et Délimitation des Quartiers

- **Fonctionnalité** : Définition visuelle des zones géographiques.
- **Description** : Intégration de la librairie Leaflet pour fournir un outil de dessin interactif. L'administrateur peut tracer les limites géographiques de nouveaux quartiers directement sur une carte. L'interface inclut des outils de validation pour prévenir et résoudre les problèmes de chevauchement ou de limites entre différentes zones.

### 2. Gestion des Habitants (CRUD Utilisateurs)

- **Fonctionnalité** : Administration de la base d'utilisateurs.
- **Description** : Un tableau de bord complet permettant de créer, consulter, modifier ou supprimer (CRUD) les profils des habitants. Les administrateurs peuvent filtrer les résidents par quartier et gérer leurs rôles (habitant, modérateur, administrateur).

### 3. Gestion des Services et Annonces (CRUD)

- **Fonctionnalité** : Modération des services.
- **Description** : Interface de supervision du système de petites annonces. L'administrateur peut consulter, éditer ou supprimer les offres et demandes de services, qu'elles soient gratuites ou basées sur le système de points.

### 4. Suivi et Gestion des Contrats

- **Fonctionnalité** : Supervision des accords entre voisins.
- **Description** : Une vue dédiée aux contrats obligatoires générés lors de services payants. L'administrateur dispose des droits de consultation (visionnage des PDF), de création (génération manuelle si besoin) et de suppression (en cas de litige ou d'erreur), tout en respectant les normes d'archivage des documents signés.

## 5. Gestion de la Vie de Quartier (CRUD Événements)

- **Fonctionnalité** : Animation de la communauté.
- **Description** : Interface permettant d'administrer les activités communautaires (soirées, ateliers, etc.). L'administrateur peut créer de nouveaux événements officiels, ou valider/modifier/supprimer les événements proposés par les habitants.

## 6. Portail de Connexion et Pont SSO

- **Fonctionnalité** : Authentification unifiée et transition vers le client lourd.
- **Description** : La plateforme web intègre le portail Single Sign-On (SSO). Depuis le back-office React, un accès sécurisé permet à l'administrateur de lancer et de s'authentifier de manière transparente vers l'application Desktop Java. Cette action est protégée par une MFA.

### *Description Fonctionnelle : Front-End Utilisateur (React)*

L'interface utilisateur est conçue pour être immersive, communautaire et hautement personnalisable. Elle repose sur un système de Dashboard modulaire et une intégration fluide avec n8n pour les processus contractuels.

### 1. Page de Connexion et Authentification

- **Fonctionnalité** : Accès sécurisé à la plateforme.
- **Description** : Interface d'authentification par adresse électronique et mot de passe. Inclut une gestion de la récupération de compte et une vérification de la localisation lors de l'inscription pour garantir que l'utilisateur appartient bien au périmètre du quartier.

### 2. Page d'Accueil (Dashboard avec widgets)

- **Fonctionnalité** : Vue panoramique et personnalisée de la vie du quartier.
- **Description** : Interface composée de widgets que l'utilisateur peut organiser.
  - **Carte interactive** : Localisation des services et événements cliquables.
  - **Mini Calendrier** : Récapitulatif visuel des activités qui arrivent.

- Vote : Accès aux derniers vote en cours
- Mission : Suggestion de missions basées sur les compétences de l'utilisateur (via Neo4j) pour gagner des points.
- Alerte Contrats : Notifications pour les documents en attente de signature.

### **3. Page « Ma Carte »**

- **Fonctionnalité** : Exploration géographique.
- Description : Utilisation de Leaflet en plein écran pour visualiser le quartier. Un système de filtres permet d'isoler les catégories (exemple : bricolage, fêtes, entraide) et d'afficher les limites du secteur.

### **4. Page « Mes Services et Contrats »**

- **Fonctionnalité** : Gestion des services et des contrats.
- Description :
  - Catalogue : Catalogue des annonces avec filtres par catégories et par prix (points ou gratuit).
  - Création de services : Formulaire intelligent pour publier une offre ou une demande (avec validation des admins)
  - Workflow de Signature (N8N) : Lecture du PDF généré automatiquement en fonction du service.
    - Interface de signature manuscrite ou d'initiales.
    - Double Authentification (2FA) : Validation de la signature via un code sécurisé envoyé par n8n par mail ou par message.
  - Gestionnaire d'Archives : Consultation classée (Passé / En cours / À venir) des contrats.

### **5. Page « Évènements et Activités »**

- **Fonctionnalité** : Animation de la vie sociale.
- Description :
  - Calendrier : Vue chronologique des événements. Les suggestions sont personnalisées selon les intérêts de l'habitant (Neo4j).
  - Créateur d'Ateliers : Outil de planification incluant une limite de participants. Chaque création est soumise à un workflow de



validation automatique ou manuelle par l'administrateur avant publication.

## **6. Page « Messagerie »**

- **Fonctionnalité** : Communication sécurisée entre voisins.
- Description : Système de chat en temps réel permettant l'échange de message. Un historique sécurisé est conservé.

## **7. Page « Vote »**

- **Fonctionnalité** : Vote participative.
- Description : Espace dédié à la démocratie locale. L'utilisateur peut consulter les descriptions des projets, voter de manière sécurisée et visualiser les résultats en temps réel via des graphiques dynamiques (Chart.js / Recharts).

## **8. Page « Classement »**

- **Fonctionnalité** : Valorisation de l'engagement au sein du quartier.
- Description : Tableaux de bord affichant les voisins les plus actifs. Le classement se base sur le nombre de services rendus, la participation aux votes ou l'organisation d'événements, avec des récompenses.

## **9. Page « Profil et Confidentialité »**

- **Fonctionnalité** : Gestion de l'identité et de la réputation.
- Description :
  - Paramètres de Visibilité : L'utilisateur choisit quelles activités sont publiques ou privées.
  - Système de Réputation : Affichage des notes (étoiles) et des commentaires laissés par les autres voisins après la réalisation d'un service.
  - Préférences : Gestion de la langue et des notifications.



## Fonctionnalités détaillées par interface :

*Description Fonctionnelle : Client Java Desktop (JAVA)*

Mise en place d'une application Desktop Java permettant de gérer les incidents et alertes signalés par les voisins via l'application WEB, ainsi qu'un aperçu des statistiques de participation.

Les données sont enregistrées dans une BDD locale embarquée afin que l'application fonctionne même sans connexion (mode Offline-First) en respectant la réglementation RGPD.

Il est possible d'ajouter et de consulter les données. Une synchronisation automatique est effectuée dès que internet est disponible avec gestion des conflits.

### **1. Gestion de plugins (export de statistique, analyse sociale, calendrier)**

- **Fonctionnalité** : Extension des fonctionnalités de l'application.
- **Description** : L'application intègre un système de plugins permettant l'export de statistiques pour générer des rapports, l'analyse sociale pour étudier l'activité des habitants dans le quartier.

### **2. Personnalisation via des thèmes (Couleurs, polices, disposition)**

- **Fonctionnalité** : Personnalisation de l'interface utilisateur.
- **Description** : L'administrateur peut modifier l'apparence de l'application grâce à un système de thèmes, il est possible de changer les couleurs de l'interface, les polices ainsi que l'organisation des éléments à l'écran afin d'adapter l'application aux préférences de l'utilisateur.

### **3. Mise à jour automatique via un serveur central**

- **Fonctionnalité** : Mise à jour automatique de l'application.
- **Description** : L'application vérifie régulièrement la présence de nouvelles version sur un serveur central. Lorsqu'une mise à jour est disponible elle peut être téléchargée et installée automatiquement.

#### **4. Désinstallation depuis l'interface utilisateur**

- **Fonctionnalité** : Gestion de la suppression de l'application.
- **Description** : L'application propose une option permettant à l'utilisateur de lancer sa désinstallation directement depuis l'interface.

## 2 – Présentation de l'infrastructure et des Technologies

Cette section détaille la stack technique, en mettant en évidence les technologies imposées par le cahier des charges et des propositions d'amélioration :

### 1. Conception & Frontend (Interface Utilisateur)

**Maquettage [Bonus/Ajout]** : Utilisation de **Figma** pour créer les frames, définir l'UX/UI et le design system avant le développement.

**Framework Web : React** pour le site utilisateur et le back-office administrateur.

### 2. Backend & Orchestration

**Serveur Applicatif : Node.js avec Express** pour l'API REST/GraphQL principale.

**Automatisation [Bonus/Ajout]** : Intégration de **N8N** pour l'automatisation des processus complexes, notamment la génération dynamique des contrats PDF à partir des annonces de services.

**Interrogation de données** : Création d'un langage d'interrogation maison via lex/yacc pour manipuler les documents MongoDB.  
**[Bonus/Ajout]** : Utilisation de **Python** (via des bibliothèques comme *PLY - Python Lex-Yacc* ou *Lark*) pour implémenter ce parseur/lexeur.

### 3. Persistance des Données (Bases de données multi-modèles)

**MongoDB** : Stockage orienté documents pour les contrats, signatures, événements et historiques de messages.

**Neo4j** : Base de données orientée graphes pour la modélisation sociale (qui aide qui) , le système de recommandations d'activités et la cartographie des routes.

**PostgreSQL** : Base de données relationnelle centrale pour la synchronisation des données structurées avec l'application Desktop.

**H2 Database** : Base de données relationnelle pour l'application Desktop. Elle permet de stocker localement les incidents et les statistiques en mode offline-first.

#### 4. Client Lourd (Application Desktop Java)

**Technologies : Java et JavaFX** pour l'interface graphique de l'administrateur

**Base de données : H2 Database** pour le stockage en local plutôt que SQLite car il est écrit en Java ce qui sera plus facile pour nous pour l'implémenter pour notre application.

#### 5. Sécurité, DevOps et Tests

**Sécurité des accès** : Mise en place du **SSO** (Single Sign-On) **Keycloak** pour faire le pont entre le web et le client Java, et d'une authentification **MFA** (Multi-Factor Authentication) pour les actions sensibles comme la suppression d'un compte pour l'administrateur ou encore la signature d'un contrat entre deux voisins.

**Déploiement & Conteneurisation** : Isolation des briques applicatives. **[Bonus/Ajout]** : Utilisation de **Docker** pour la création des conteneurs et de **Kubernetes** pour l'orchestration, garantissant l'extensibilité de la plateforme.

**Qualité logicielle** : Implémentation de tests unitaires, E2E et d'intégration, tant sur l'écosystème Node/React que sur l'application Java.

## 3 – Charte graphique

Couleurs primaires



#247276



#F29C55



#FCD6A7



#8AC6C7

Couleurs Secondaires

**Quicksand**

# PetitsSecretsEntreVoisins

PetitsSecretsEntreVoisins

**Montserrat**

# Petits Secrets Entre Voisins

PetitsSecretsEntreVoisins

**Liens :**

Github : <https://github.com/fayeza78/ProjetAnnuel>

Trello : <https://trello.com/b/WPLVOEKs/projet-annuel-petit-secrets-entre-voisins>